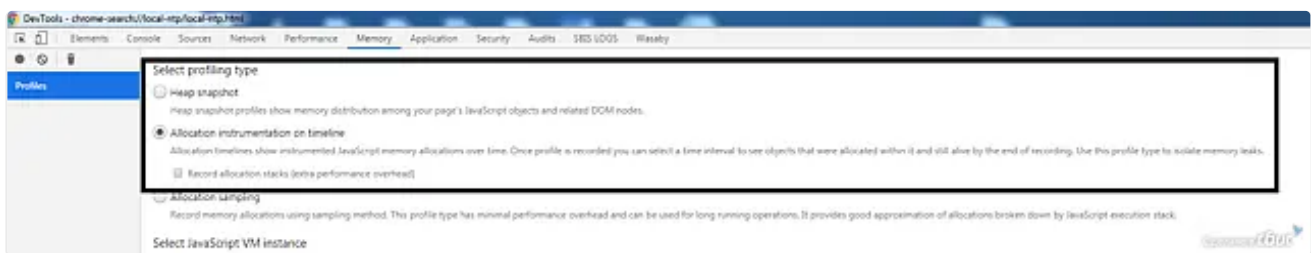


Методика тестирования действия на предмет утечки памяти

Для тестирования утечек памяти у Chrome есть удобный и наглядный инструмент — вкладка **Memory** в DevTools. Мы используем два режима записи памяти:

- **Снапшот** (Heap snapshot) — мгновенный статичный слепок кучи и связанных с ней DOM-объектов.
- **Таймлайн** (Allocation instrumentation on timeline) — продолжительная динамичная запись кучи и связанных с ней DOM-объектов. В первое мгновение записываются все созданные до старта объекты. Далее начинается запись во времени изменения кучи и DOM. Если создается объект, он ставится в свою временную отметку, в которую он создан. Если объект удален — он удаляется из таймлайна. Соответственно, показывается актуальный объем занимаемой памяти. Вместе с таймлайном можно записать стек для каждого созданного объекта, но это требует намного больше ресурсов компьютера (Record allocations stacks).



Пример анализа

Разберём поиск утечки на конкретном примере. Возьмём такой код:

▼ Пример кода

```
1  const moduleArray = [];  
2  
3  function generateRandomSequence() {  
4      const result = [];  
5      for (let i = 0; i < 10000; i++) {  
6          result.push(Math.random());  
7      }  
8      return result;  
9  }  
10 function modulePush() {  
11     moduleArray.push(generateRandomSequence());  
12 }  
13  
14 document.getElementById("modulePush").addEventListener("click",  
    modulePush);
```

При каждом нажатии кнопки в массив moduleArray будет добавляться 10000 случайных чисел.



По [ссылке](#) можно открыть страницу, на которой все действия из примера можно повторить самостоятельно.

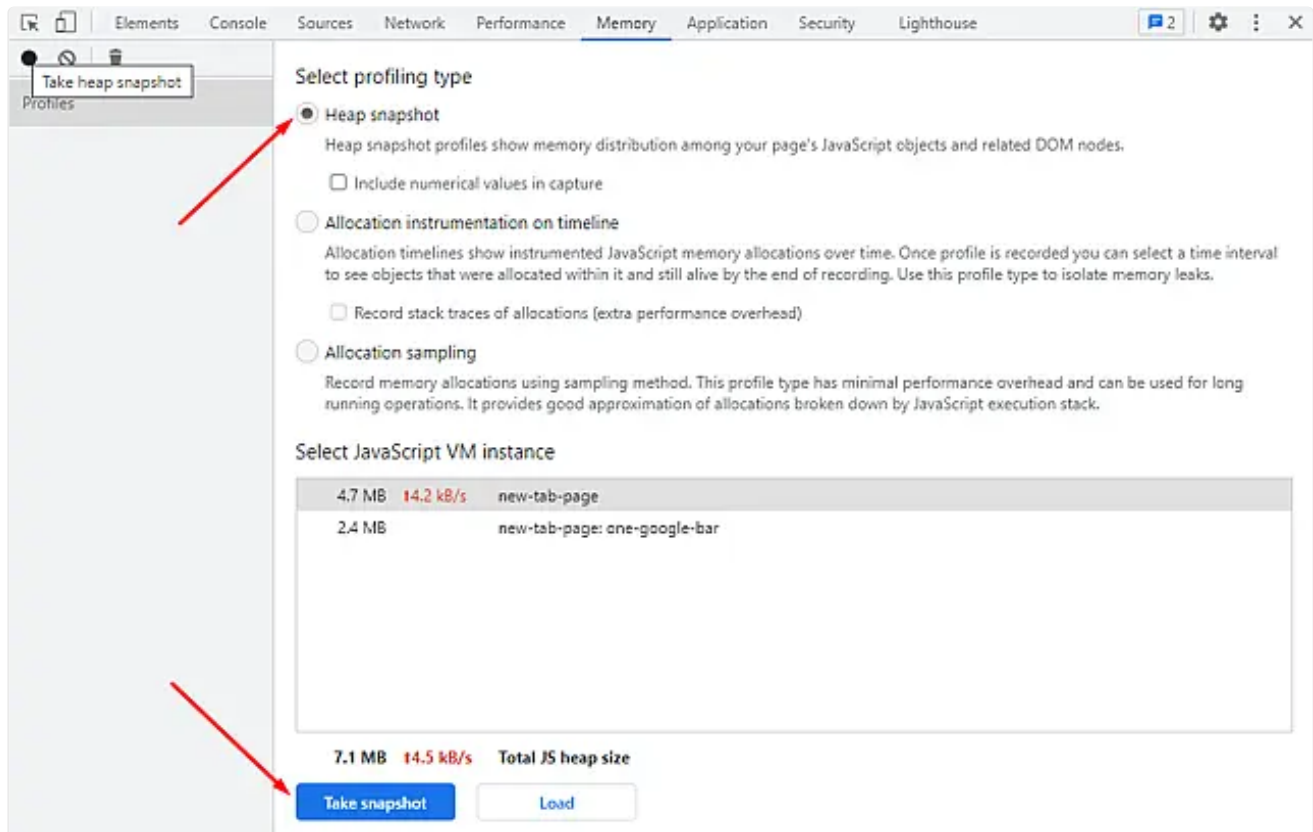
Подготовка к тестированию

1. Открываем последний стабильный Chrome в режиме инкогнито без плагинов.
2. Отключаем режим отладки с помощью удаления куки-файла s3debug=true и перезагружаем. В режиме отладки анализировать объекты проще, но определять наличие утечки корректно только без куки, так как с кукой меняются некоторые алгоритмы исполнения скриптов.

3. Во время замера нельзя использовать DevTools-консоль (выполнять в ней свой код) или же отладчиком (использовать точки останова), так как они могут удерживать объекты в памяти. Вопрос о том, надо ли очищать консоль при замере пока спорный, так как пользователь консоли не чистит, а удерживает ли консоль объекты при закрытом DevTools надо исследовать отдельно.
4. Определяем тест кейс. Он должен состоять из тестируемого действия и **обязательно** возврата к начальному состоянию. Также в кейсе не должно быть перезагрузки страницы.
5. 1-2 раза выполняем наш тест-кейс на странице без записи.

Тестирование с использованием снапшотов

Для начала нажмём кнопку "Добавить элемент в массив, объявленный на уровне модуля" пару раз и снимем слепок. Для этого нужно выбрать пункт "Heap snapshot" и нажать кнопку "Take snapshot":



После этого добавим ещё несколько элементов в массив и снимем второй слепок тем же способом. Получится примерно такая картина:

1. Выбираем интересующий нас элемент.
2. Смотрим в "Retainers". Идём по цепочке объектов и для каждого пытаемся понять должен ли он быть в памяти. Наша задача — найти самый близкий к корню элемент, которого не должно быть в памяти.
3. После этого остаётся понять причину удержания объекта и устранить её.

Как понять, какой элемент нас интересует? Для этого разберём столбцы верхней части таблицы:

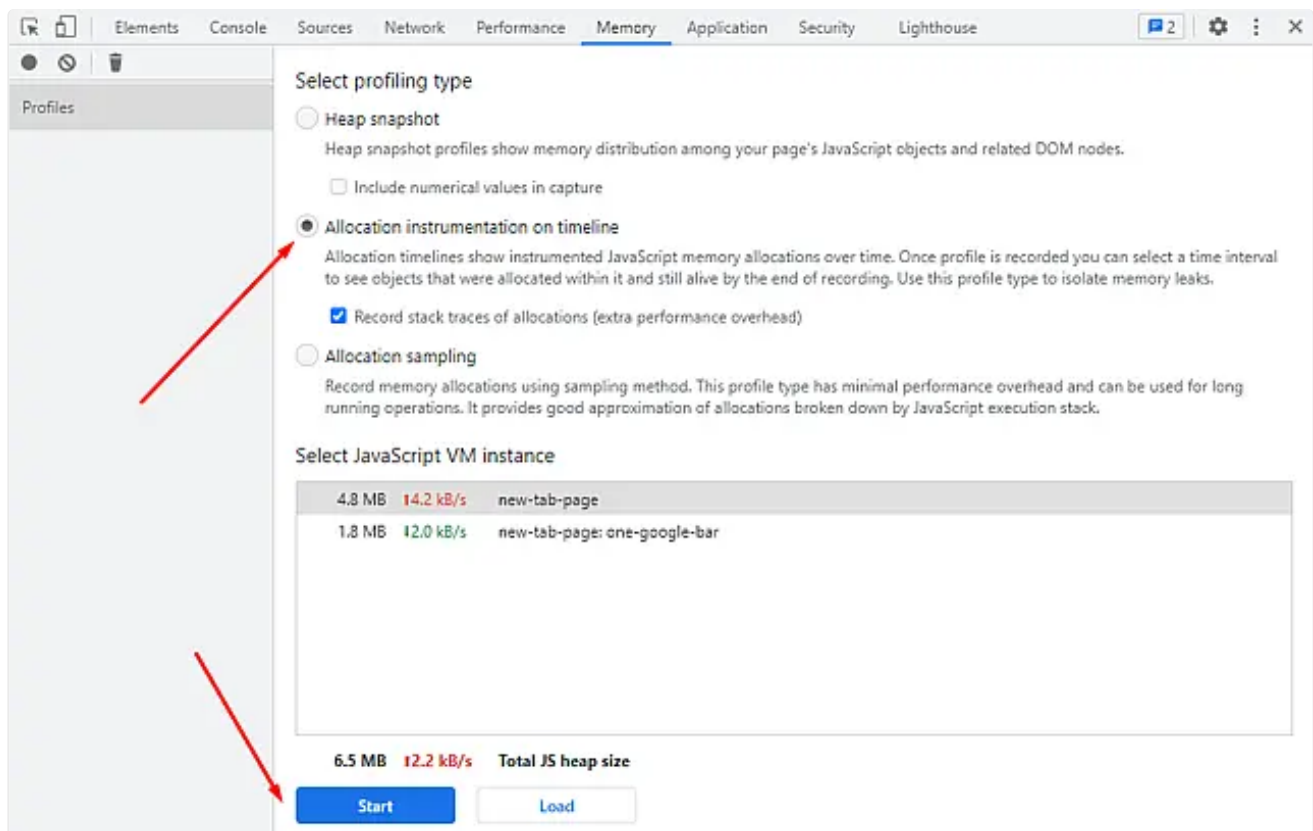
1. Constructor — название конструктора элемента. Обычно нас волнуют вещи сложные типа, например, Array или Object, пользовательские конструкторы типа Record/Control, а также любые разделы, содержащие слово "detached".
2. Distance — расстояние от глобального объекта. Обычно стоит начинать поиски с самого большого расстояния и подниматься вверх. Нас никогда не волнует самый верхний элемент, потому что он никого не держит в памяти. Самые нижние элементы нас не интересуют потому что там слишком много шума. Реальная утечка всегда находится где-то между ними.
3. Shallow size — объём памяти, занимаемый объектом. Обычно эта колонка не даёт полезной информации при отладке утечек.
4. Retained Size — объём памяти, занимаемый объектом + объём памяти, занимаемый объектами, которые он удерживает. Самая важная колонка — чем больше цифра, тем больше утечка, поэтому имеет смысл начинать с самых больших значений.

В данном примере видно, что выбранный элемент находится в массиве moduleArray, который удерживается в памяти функцией modulePush, которая удерживается обработчиком событий кнопки.

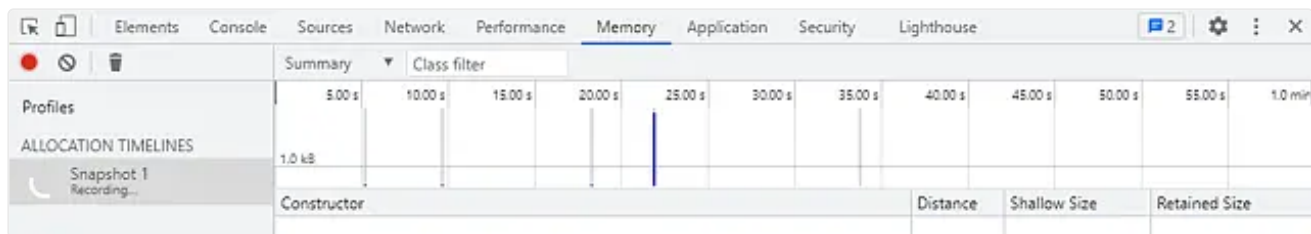
Тестирование с использованием таймлайна

Разберём второй тип профилирования: "Allocation instrumentation on timeline". При данном типе профилирования записываются все операции с памятью за время профилирования. Если поставить галочку на "Record allocation stacks", то для каждой операции запишется стек вызовов.

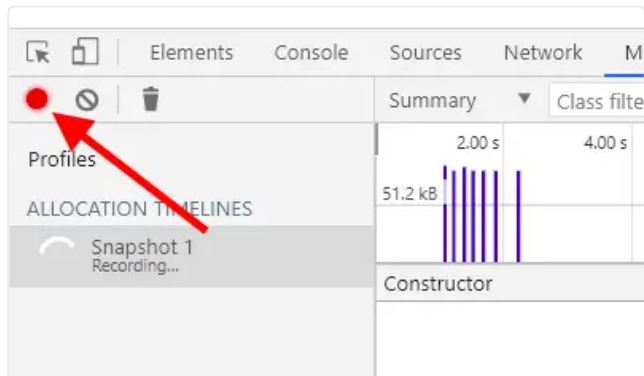
Перезагрузим наш пример, чтобы очистить память и начнём профилирование. Для этого нужно выбрать пункт "Allocation instrumentation on timeline" и нажать кнопку "Start":



Обратите внимание, что при каждом добавлении элемента в массив в верхней части экрана появляются столбики:



Эти столбики обозначают выделение памяти. Добавим несколько элементов в массив и остановим запись с помощью кнопки в левом верхнем углу инструментов разработки.

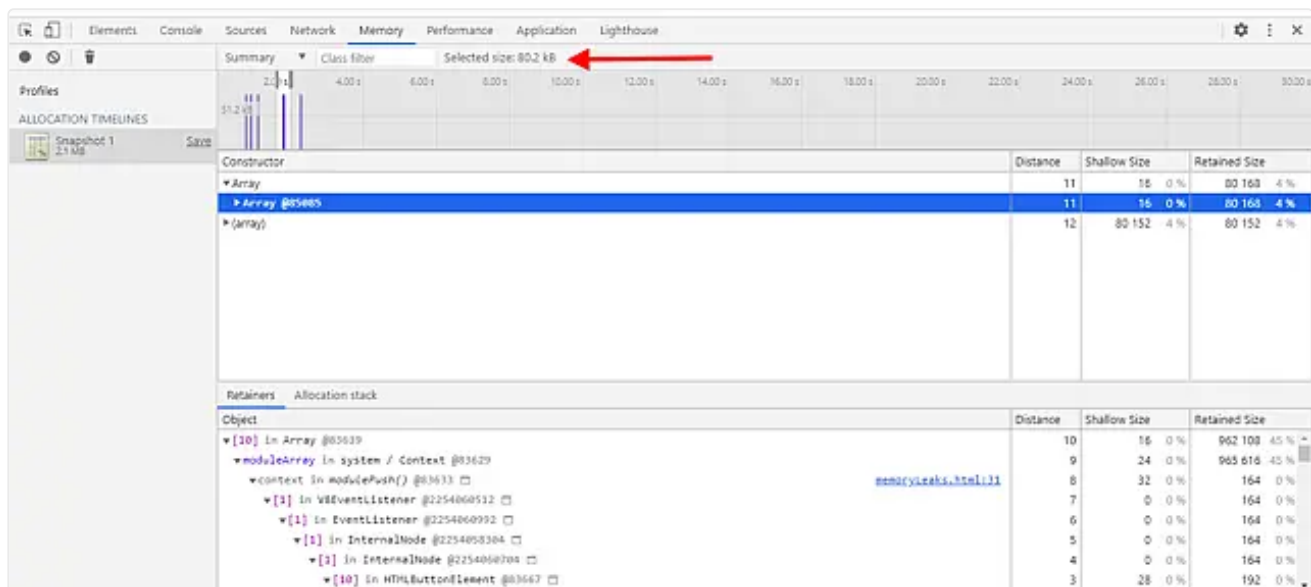


В результате мы получим профиль, который будет выглядеть почти также, как предыдущий, но в верхней части добавился таймлайн. Каждый столбик может состоять из двух цветов:

- Синий — объём памяти, которая не была собрана сборщиком мусора.
- Серый — объём памяти, которая была собрана сборщиком мусора.

Соответственно, нас волнуют синие части. Если мы выделим одну из них с помощью мыши, то в таблицах останется только информация об объектах, относящихся к этому выделению. Для анализа лучше выбирать область из середины, чтобы отсеять шум.

Обратите внимание на надпись "Selected size: 80.2 kB" справа от поиска, она показывает, сколько конкретно памяти удерживается выделенными объектами.



Дальше следуем по тому же алгоритму, что и в прошлый раз — выбираем объект и ищем, кто его удерживает.

Алгоритм поиска утекающих объектов на сервисе представления

Процесс поиска утечек на сервере схож с поиском утечек на клиенте. Отличие только в том, что отлаживаться нужно через подключение к сервису представления (см. [руководство](#)).